Google Summer of Code

JSON Schema

Project Proposal

Studio-Json-Schema

GSOC 2026: Enhanced Interaction and Navigation in JSON Schema Studio

Mentors : Mr. Agnivesh Chaubey

Personal Details:

Name: Animesh Sharma
Email: Animesh Sharma
Github: Animesh-86
Slack: Animesh
Time Zone: IST (GMT + 5:30)

I am a 3rd-year Computer Science student with a strong focus on developer tooling, distributed systems, and schema-driven architecture. I recently implemented a JSON parser from scratch in Java to deepen my understanding of structured data parsing and schema validation, which naturally led me to explore the JSON Schema ecosystem and tools like JSON Schema Studio.

Why this Project ?

JSON Schemas used in real-world applications can be complex and hard to navigate, and it can be difficult for developers to understand the relationships between various parts of a schema. This can be solved by tools such as JSON Schema Studio, where a schema can be visually represented to make it easier for developers to debug. The aspects of the project that I find most interesting include the potential to improve the way developers can interact with complex schemas. The aspects of the application that I believe need improvement include the dependency ordering, graph navigation, and synchronization of the schema. These are important aspects of the application that can be improved to make it much easier for developers to use.

Technical Project

My engineering focus is on frontend architecture, developer tooling, and building highly interactive user interfaces using React and TypeScript. I primarily work with JavaScript/TypeScript, Java, and Python, and have experience building full-stack applications using modern technologies such as React, Spring Boot, and Flutter. My development experience includes designing scalable systems, creating developer tools, and working extensively with structured data formats such as JSON.

In order to better understand data formats and their corresponding systems, I implemented a JSON parser from scratch using Java, in which a tokenizer, recursive parser, and internal representation were created to correctly parse JSON objects, arrays, and primitive values. This gave me a better idea of how structured data is internally parsed and validated, naturally leading me to JSON Schema and tools like JSON Schema Studio.

Some of my notable projects are:

• Axion – EV Fleet Digital Twin & OTA Orchestration Platform
Axion is a distributed system designed to simulate the ingestion of EV fleet telemetry and management of digital twins, using backend microservices.

• Parallax – Real-time Collaborative Coding Platform
Parallax is a collaborative coding platform developed using React, TypeScript, Monaco Editor, and Spring Boot, enabling multiple users to edit code in real-time using WebSockets.

• Cafe POS System (Flutter + Firebase)
The Cafe POS System is a mobile application developed for a real café using Flutter and Firebase, enabling billing, inventory management, and order management with cloud syncing.

I have gained proficiency in the following areas through these activities:

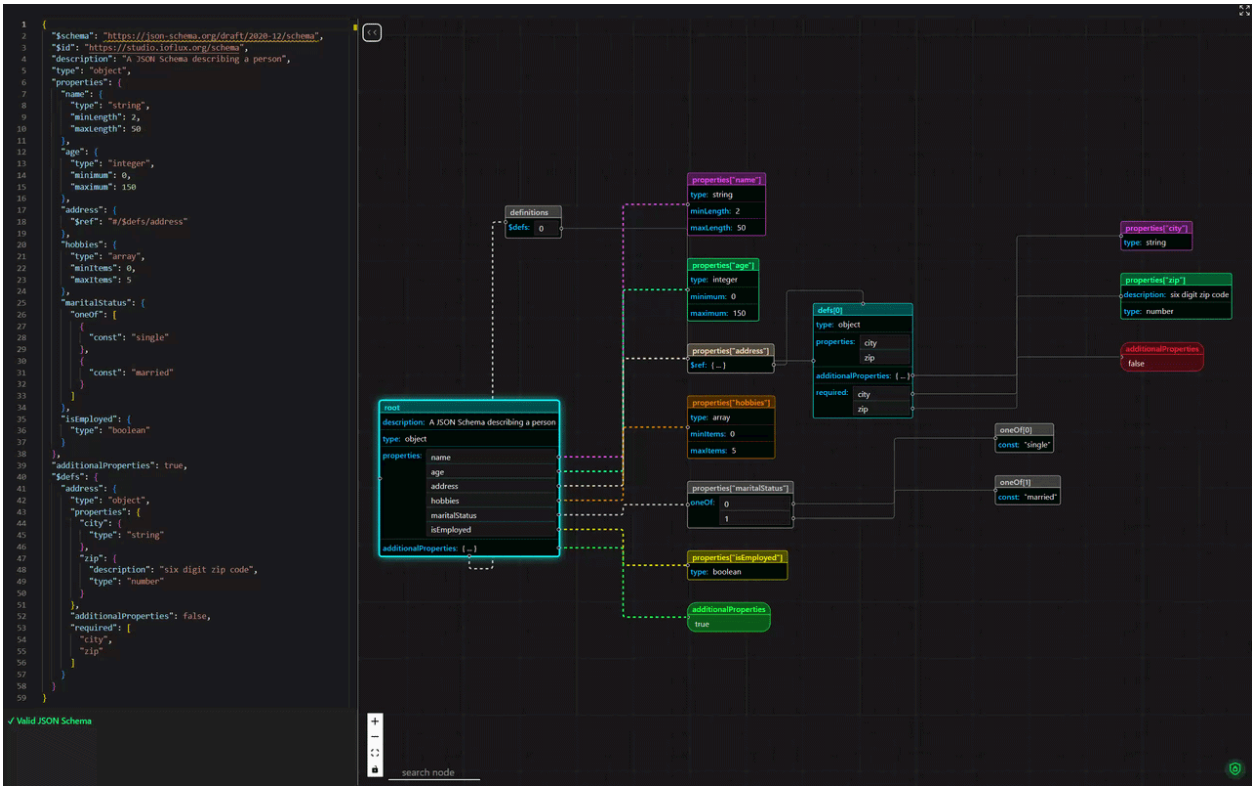
• Java, SpringBoot, JavaScript / TypeScript Development
• React-based frontend development
• Data processing and JSON systems
• Use of the Git workflow
• Debugging and navigating through large codebases

These skills match the technologies used in JSON Schema Studio, especially React-based visualization systems and schema parsing systems.

Project Understanding:

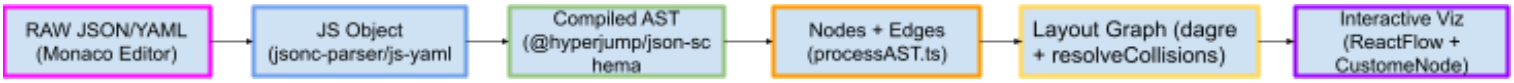
What is JSON Schema Studio?

JSON Schema Studio is an open-source, browser-based tool that turns raw JSON Schema definitions into interactive directed graphs. Instead of reading through deeply nested JSON to understand a schema's structure, you get a color-coded visual map showing how properties, types, references, and constraints relate to each other. You type on the left, and the graph updates live on the right.



The Pipeline: How a Schema Becomes a Graph

This app follows a 4 stage pipeline. I traced each stage through the code to understand the full flow



Stage 1: Schema Input (MonacoEditor.tsx)

The Monaco Editor accepts both JSON and YAML. When the user types, the input is parsed:

- JSON - `jsonc-parser` (supports comments and trailing commas)
- YAML - `js-yaml` (converted to a JS object, then serialized back to JSON for Hyperjump)

The editor also validates the schema in real-time, showing a green "✓ Valid JSON Schema" or red error messages at the bottom-left.

Stage 2: Compilation (Hyperjump)

This is where I had to spend time reading the internals. The parsed schema object gets registered with Hyperjump's `buildSchemaDocument()`, then compiled with `compile()`. What Hyperjump produces is an AST — but it's not a tree. It's a flat map where:

- Keys are schema URIs
- Values are arrays of [keywordHandlerURI, keywordURI, keywordValue] tuples

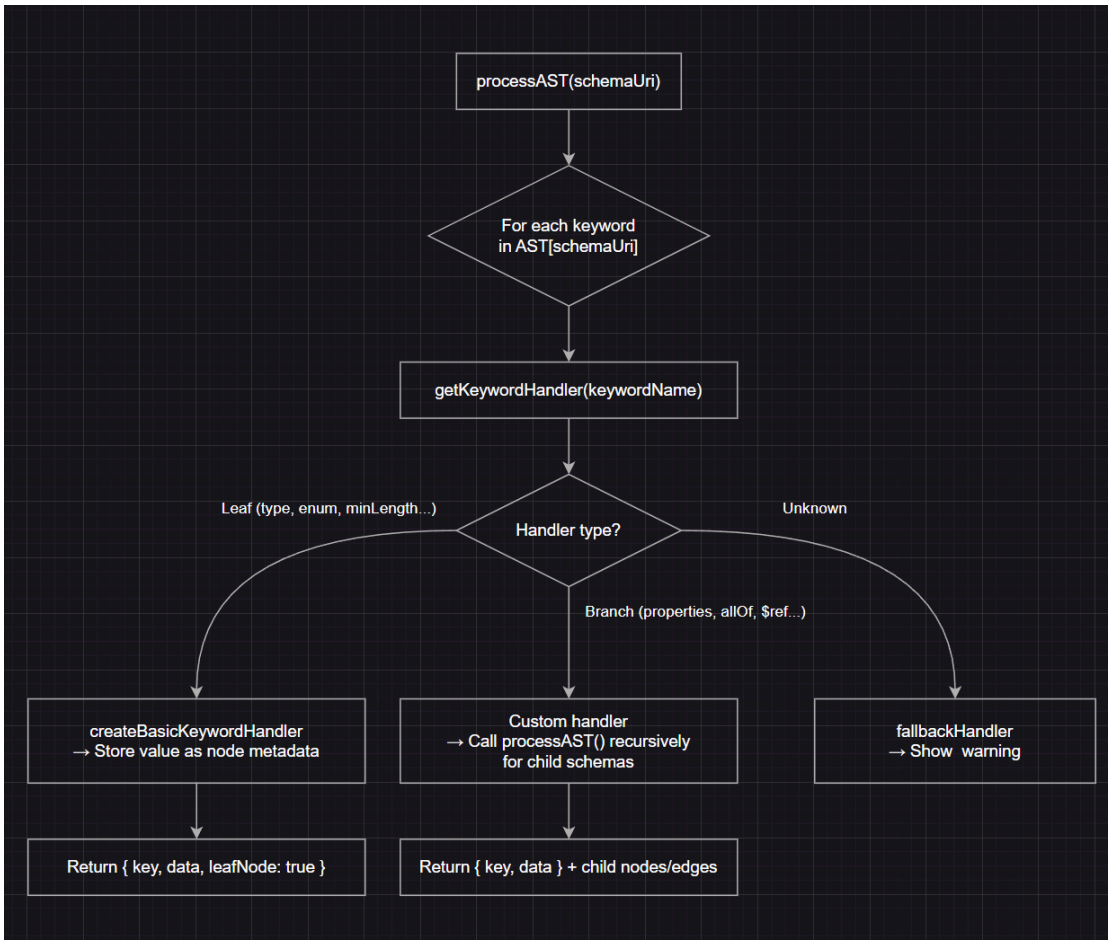
For example, the name property in the default schema produces something like:

```
"https://studio.ioflux.org/schema#/properties/name": [
  ["https://json-schema.org/keyword/type", "...", "string"],
  ["https://json-schema.org/keyword/minLength", "...", 2],
  ["https://json-schema.org/keyword/maxLength", "...", 50]
]
```

This flat structure is important, it means that file processAST has to recursively discover the graph structure by following keyword values, not by walking a pre-built tree,

Stage 3: AST → Nodes & Edges (processAST.ts)

This is the heart of the visualization `processAST()` walks the AST recursively and builds ReactFlow compatible nodes and edges. It uses a `keyword handler pattern`, each JSON Schema maps keyword maps to a handler function in keywordHandlerMap:



What I found interesting here:

- Leaf keywords (like `type`, `minLength`, `enum`) use createBasicKeywordHandler, a factory that just stores the value as node metadata without creating child nodes
- Branch keywords (like `properties`, `allOf`, `oneOf`, `$ref`) have custom handlers that recursively call processAST for each child schema, creating, new nodes and edges
- Circular references are handled via a `renderedNodes` Map, if schema URI was already processed, it creates a back edge to the existing node instead of duplicating the subtree
- If the keyword has no handler, a `fallbackHandler` catches it and displays "not implemented", warning in the node. I noticed that dependentSchemas is currently commented out at line 353 that is where my proposed handler will go.

Stage 4: Layout and Rendering

The raw nodes (which have no position yet) got through two layout passes:

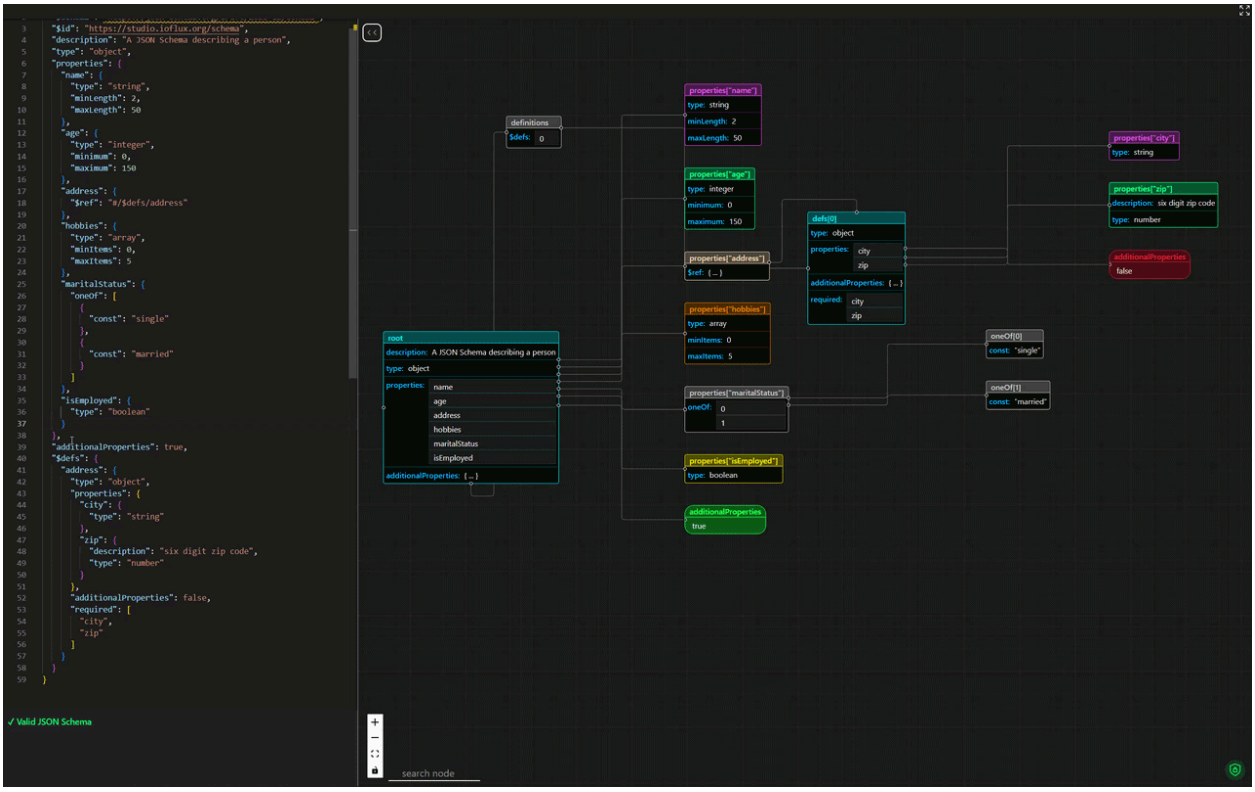
- Dagre (`@dagrejs/dagre`) arranges nodes in a left to right directed graph, representing parent child relationship via edges.
- Collision resolution (`resolveCollision`) iteratively nudges overlapping nodes apart. This runs after ReactFlow measures actual node dimensions (since nodes vary based on content)

ReactFlow then renders everything with `CustomerReactFlowNode.tsx`, which displays node as card with:

- A colored header showing the property name
- A table of keyword → value pairs
- Source handles (right side) for child connections
- Target handles (left side) for parent connections

Real-time Updates

When the schema is edited, the entire pipeline re-runs → parse → compile → processAST → layout → render, Here's a recording showing this in action - i added and email property with format' "email" and the graph updated instantly:



Where my improvement fits

The keyword handler architecture is designed to be extensible. Adding support for `dependentSchemas` means adding a new entry to `keywordHandlerMap` at the exact line where it's currently commented out (line 353 of processAST.ts). The handler will follow the same pattern as properties — iterate over the keyword value entries and recursively call processAST for each dependent schema.

For topological sorting of `$defs`, the improvement fits in the gap between Stage 2 (AST compilation) and Stage 3 (processAST traversal). Currently, $defs are rendered in whatever order they appear in the AST. By analyzing dependency chains between definitions before rendering, we can ensure that referenced definitions are rendered before their referrers, producing a cleaner, more logical graph layout.

Contribution & Engagement with JSON Schema Studio

Before preparing this proposal, I spent time exploring the codebase and actively contributing — not just analyzing, but actually writing code, debugging issues, and understanding how each piece connects. Here's what I worked on:

1. Extend Search to Include nodeData Values (#134):

While testing the search feature, I tried searching for "string", expecting to find all nodes with type: string. Instead, the search returned nothing. The search only matched against node labels (properties["name"], defs[0]) using extractKeywords(), but completely ignored the keyword values stored inside each node's metadata.

```
// Before: only searched node labels

const foundNodes = nodes.filter((node) => {

  const labelWords = extractKeywords(node.data.nodeLabel);

  return searchWords.every((word) => labelWords.some((w) =>
w.includes(word)));

});
```

To fix this, I wrote extractNodeDataValues() in searchNodeHelpers.ts that pulls searchable text from the nodeData object, extracting both the keyword names (type, minLength) and their values (string, 2, 50): and traced the matching logic:

```
// After: searches both labels and keyword values
const foundNodes = nodes.filter((node) => {
  const labelWords = extractKeywords(node.data.nodeLabel);
  const valueWords = extractNodeDataValues(node.data.nodeData);
   const allWords = [...labelWords, ...valueWords];
  return searchWords.every((word) => allWords.some((w) =>
w.includes(word)));
});
```

Now searching "string" correctly returns nodes like properties["name"] (because it has type: string) and properties["city"], which is what users would intuitively expect.

2. Node Color Defaulting to Gray — Bug Discovery & Fix (#136):

I noticed that when a schema had nodes with only a type keyword and no type-specific constraints (like minLength or minimum), all nodes rendered as gray (#CCCCCC) instead of their correct type-based colors.

I traced the issue through the rendering pipeline: processAST.ts calls inferSchemaType() to determine the node color. Inside inferSchemaType.ts , I found the bug — a type comparison that could never succeed:

```
// BUG: nodeData.type is a NodeDataValue object like { value:
"string" }
// typeof { value: "string" } === "object", NEVER "string"
if (typeof nodeData.type === "string") {
    return ["objectSchema", nodeData.type];  // never reached
}
```

Because this condition always failed, the function fell through to keyword-based inference. If there were no type-specific keywords like minLength or minimum, it returned "others" → gray.

The fix was straightforward — check .value instead of the wrapper object:

```
// FIX: check the actual .value property

if (nodeData.type && typeof nodeData.type.value === "string") {

    return ["objectSchema", nodeData.type.value];

}
```

I opened the issue with full root cause analysis, reproduction steps, and a PR with the fix. The maintainer later independently created the same issue, and another contributor's PR for the same fix was merged — but I had already identified the root cause, traced it to the exact line, and understood the underlying NodeDataValue wrapper pattern that caused it.

3. Enhanced YAML Syntax Highlighting (#98):

The YAML editor was using Monaco's default theme, which made everything look the same color — keys, values, numbers, and booleans were nearly indistinguishable, especially in larger schemas.

I built a custom YAML theme system from scratch:

- Created src/utils/monacoThemes.ts with custom tokenizer rules for YAML tokens
- Connected it to MonacoEditor.tsx using the beforeMount hook (so themes are registered before the editor renders, avoiding a flash of unstyled content)
- Built separate color schemes for both dark and light modes

The color choices were deliberate:

- Keys → purple (dark) / brown (light) — bold enough to show structure at a glance
- String values → red — stands out from keys immediately
- Numbers & booleans → blue — visually grouped as "data values"
- Comments → gray italics — de-emphasized, as they should be
- Delimiters → soft gray — fade into the background

I posted before/after screenshots for both modes showing the improvement in readability. The branch is ready for a PR once the maintainer reviews the approach.

What These Contributions Taught Me:

Each contribution forced me deeper into a different part of the codebase:

- #134 → taught me how nodeData is structured as Record<string, NodeDataValue> and how the search pipeline flows from input → word extraction → node filtering
- #136 → taught me the full type inference chain: processAST → inferSchemaType → neonColors, and the critical distinction between raw values and NodeDataValue wrapper objects
- #98 → taught me Monaco's theming API, how the editor integrates via lifecycle hooks, and how the JSON/YAML format toggle works end-to-end

This hands-on experience is what gave me the confidence to tackle the qualification task, especially understanding how keyword handlers create nodes and how data flows from the Hyperjump AST through processAST into the visualization layer.

Qualification Task

Task 1: Dependent Schemas Handler

What dependent schema does:

It's a conditional applicator: "If property X exists in the data, the entire object must also satisfy this additional subschema."

```
{
  "type": "object",
  "properties": {
    "name": { "type": "string" },
    "creditCard": { "type": "string" }
  },
  "dependentSchemas": {
    "creditCard": {
      "properties": {
        "billingAddress": { "type": "string" }
      },
      "required": ["billingAddress"] }}}
```

If creditCard is present → the object must also have billingAddress as a required string.

How I figured out the AST structure:

I went into `node_modules/@hyperjump/json-schema/lib/keywords/` and read the actual source code for `dependentSchemas.js`, `properties.js`, and `patternProperties.js`. The key finding:

```
// properties.js collects into an OBJECT
asyncCollectObject  - { name: "schemaURI", age: "schemaURI" }
// dependentSchemas.js collects into an ARRAY of tuples
asyncCollectArray   - [["creditCard", "schemaURI"], ...]
// patternProperties.js also collects into an ARRAY
asyncCollectArray   - [[RegExp, "schemaURI"], ...]
```

The Handler:

```
"https://json-schema.org/keyword/dependentSchemas": (ast,
keywordValue, nodes, edges, parentId, nodeDepth, renderedNodes)
=> {
    const value = keywordValue as [string, string][];
    const dependentPropertyNames: string[] = [];
    for (const [propertyName, schemaUri] of value) {
        dependentPropertyNames.push(propertyName);
        processAST({
            ast,
            schemaUri,
            nodes,
            edges,
            parentId,
            renderedNodes,
            childId: propertyName,
            nodeTitle: `dependentSchemas["${propertyName}"]`,
            nodeDepth
        });
    }
    return { key: "dependentSchemas", data: { value:
dependentPropertyNames } }
},
```

Design decisions

From `properties` I borrowed:

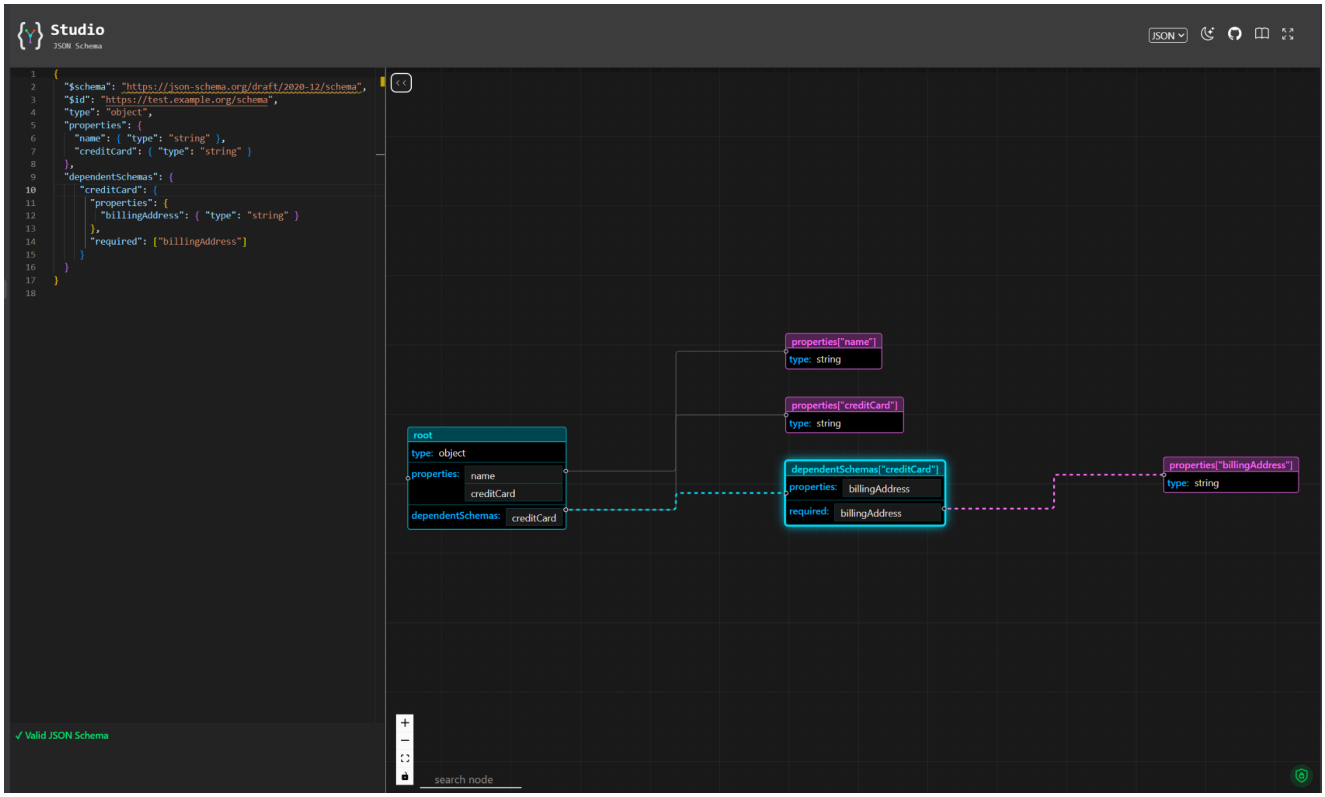
- Property names as childId → source handles become parentId-creditCard (semantically meaningful, unlike patternProperties which uses numeric indices)
- Property names in nodeTitle → dependentSchemas["creditCard"] (consistent with properties["name"])
- Collecting names into dependentPropertyNames → parent node displays the list of conditional dependencies

From `patternProperties` I borrowed:

- Casting keywordValue as [string, string][] and iterating with for...of → matches Hyperjump's asyncCollectArray output
- NOT using Object.entries() → because the value is an array, not an object

What I chose not to do:

Adding `ellipsis: "{ ... }"` to each value (like additionalProperties does) — would clutter the parent node. Showing property names directly is cleaner.



Task 2: Topological Sort for $defs Dependencies

The problem, Currently, `$defs` entries are rendered in AST order (line 289-295). When definitions reference each other:

```
{
  "$defs": {
    "fullAddress": {
      "properties": { "country": { "$ref": "#/$defs/country" } }
    },
    "country": {
      "type": "string", "enum": ["US", "UK", "IN"]
    },
    "mailingLabel": {
      "allOf": [{ "$ref": "#/$defs/fullAddress" }]}}}
```

The dependency chain is `mailingLabel → fullAddress → country`. Rendering in original order means the $ref edge from fullAddress to country points right-to-left-a backwards edge that makes the graph confusing.

Rendering in topological order `(country → fullAddress → mailingLabel)` makes every edge flow left-to-right, matching the graph's visual direction.

My approach: Kahn's algorithm with fallback insertion ordering

Phase 1 — Build the dependency graph:

Walk each definition's AST entries and check for $ref keywords pointing to sibling $defs:

For each definition D in $defs:
  For each keyword in D's AST:
    If keyword is "$ref" and target is a sibling $defs entry:
      Add edge: D depends on target

Phase 2 — Kahn's algorithm (BFS-based topological sort):

1. Calculate in-degree for each definition
2. Add all in-degree=0 definitions to a queue
3. While queue is not empty:

   a. Dequeue a definition → add to sorted output
   b. Decrement in-degree of all its dependents
   c. If any reach in-degree 0 → enqueue them
4. If sorted output has all definitions → done
   If not → cycle detected → append remaining in original order

For the example: in-degrees are country=0, fullAddress=1, mailingLabel=1. Starting from country, it processes sequentially, producing [country, fullAddress, mailingLabel].

Where it fits in the code

Inside the $defs handler, before the rendering loop:

```
"https://json-schema.org/keyword/$defs": (ast, keywordValue, ...) => {
    const value = keywordValue as string[];
    const sorted = topologicalSort(value, ast);  // new utility
    for (const [index, item] of sorted.entries()) {
        processAST({ ..., schemaUri: item, ... });
    }
    return { key: "$defs", data: { value: getArrayFromNumber(value.length)
} }
},
```

topologicalSort() would live in a new file (sortDefs.ts) — keeping processAST clean.

Why Kahn's algorithm over alternatives

Approach	Pros	Cons
DFS-Based sort	Simple recursive implementation	Recursion depth = number of defs → stack overflow risk for large schemas (200+ defs). Doesn't detect cycles until recursion finishes.
Kahn’s (BFS)	No recursion, O(V+E), cycle detection is built-in (leftover nodes = cycle), queue never exceeds number of defs	Slightly more code than DFS

Reply on dagre	No new code needed	Dagre works on final nodes, but the problem is in rendering order — renderedNodes Map determines edge directions during traversal, before dagre run
Two-pass rendering	Could re-order after all nodes exist	Requires significant refactoring of the single-pass recursive design for a problem that only affects $defs

Proposed Timeline & Technical Approach (90 Hours)

{Note: The following timelines and technical approaches are proposed drafts based on my current understanding of the codebase and project goals. I am completely open to discussing alternative algorithms, adjusting the timeline pacing, or refining these technical designs based on mentor feedback and architectural priorities.}

Since this is a 90-hour project, I plan to dedicate about 10-12 hours per week over an 8-week core coding period. I've broken down the 8 expected outcomes from the project description into logical phases that build on top of each other.

Pre-Coding & Community Bonding (May 4 – May 26)

Goal: Finalize the exact approaches with my mentors and set up my local testing environments.

Tasks:

- Discuss my Kahn's Algorithm approach for `$defs` sorting with the maintainers to ensure it fits their long-term vision.
- Flesh out the exact UI/UX design for the edge-based navigation buttons (where exactly should the buttons appear on the edge without blocking node text?).
- Continue reviewing any new PRs from other contributors to stay familiar with main

Phase 1: Editor & Canvas Synchronization (Weeks 1-2)

Focus: Fixing how the editor and the graph talk to each other during live edits and errors.

Week 1: Validation Error Navigation

Goal: Make clicking an error highlight the exact line in the Monaco Editor.

Approach: I'll map the JSON pointers returned by `@hyperjump/json-schema` (like `/properties/user/type`) to line numbers. I've looked into `jsonc-parser`, and its getLocation API will let me find the exact offset in the text. I'll then use Monaco's `editor.revealLineInCenter` to scroll the user there.

Challenge: YAML line mapping might be slightly different than JSON. I'll need to make sure the offset calculation works for both formats.

Week 2: Fix Handle Misalignment on Live Edits

Goal: Stop ReactFlow connection dots from becoming misaligned when the schema changes.

Approach: I noticed that when nodes change size (like adding a row to the property table), ReactFlow's internal state hasn't caught up yet. I plan to attach a ResizeObserver inside `CustomReactFlowNode.tsx`. Whenever the node's dimensions change, I'll trigger ReactFlow's `updateNodeInternals(nodeId)` to force the handles to snap back into place immediately.

Phase 2: Graph Navigation & Interaction (Weeks 3-4)

Focus: Making large schema graphs easier to explore without getting lost.

Week 3: Edge-Based Navigation Controls

Goal: Add UI buttons on edges to jump directly to the source or target node.

Approach: I'll replace the standard ReactFlow edges with a custom `NavigableEdge` component. Using the edge's midpoint coordinates `(labelX, labelY)`, I'll render a small `<foreignObject>` that appears on hover. Clicking these will trigger

`useReactFlow().fitView({ nodes: [{ id: targetId }] })` for a smooth pan to the destination.

Week 4: Distinct Visualization for Multi-Type Schemas

Goal: Make schemas like `type: ["string", "null"]` visually distinct from single types.

Approach: Currently, these fall back to a default gray. I'll update the logic in inferSchemaType.ts to return an array of types instead of just one. Then, in `CustomReactFlowNode.tsx`, I'll use CSS `linear-gradient` borders to blend the colors (e.g., half Magenta, half Purple) and add small UI badges next to the property name to make it explicitly clear.

Phase 3: Algorithms & Layout (Weeks 5-6)

Focus: The heavy lifting for rendering logic.

Week 5: Topologically Sorted Rendering of $defs

Goal: Ensure `$defs` are rendered without tangled, right-to-left backward edges.

Approach: As I outlined in my Qualification Task, I will implement Kahn's Algorithm (BFS) directly inside the $defs keyword handler in `processAST.ts`. I will build the dependency graph of internal `$ref` pointers first, sort them, and then pass them to `processAST`. If I detect a cycle, I'll fall back to their original insertion order so the app doesn't crash.

Week 6: Edge Collision Resolution

Goal: Reduce edge lines completely overlapping nodes.

Approach: Right now, edges just draw a straight or bezier path directly through whatever is in the way. I want to implement a custom step edge that routes around nodes. I'll use ReactFlow's state (`useStore(s => s.nodeInternals)`) to grab the bounding boxes of all nodes and use them as obstacles for a basic orthogonal routing algorithm.

Phase 4: Utilities & Polish (Weeks 7-8)

Focus: Wrapping up the remaining features and ensuring stability.

Week 7: File Upload & Export Features

Goal: Let users upload local files and download graph images.

Approach (Upload): Add a hidden <input type="file" /> to NavigationBar.tsx. Parse it with the FileReader API and push it to the editor state.

Approach (Export): I'll use the `html-to-image` library. I'll add an "Export Graph" button that captures the `.react-flow__viewport` div, generates a PNG data URI, and triggers a download. I've used this exact library with ReactFlow before, and it works flawlessly with custom nodes.

Week 8: Buffer, Testing & Documentation

Goal: Ensure everything merged is robust and documented.

Tasks:

- Write test cases for the new topological sort logic.
- Cross-browser testing for the newly added UI elements (like the edge navigation buttons and file uploads).
- Write the final GSoC report and update the project's README or docs if needed.

Post-GSoC Commitment

I don't view GSoC as a temporary gig. Because JSON Schema Studio is closely tied to the tooling I already use, I plan to stick around as an active contributor long after the official coding period ends, helping review PRs and triage issues for the new community members.

Deliverables & Milestones

The 90-hour project is structured around 8 core deliverables, mapped directly to the expected outcomes. I guarantee the delivery of all 8 features, fully tested and integrated into `main`.

Milestone 1 (Weeks 1-2): Synchronization & Consistency

- PR 1: Validation Error Navigation (Hyperjump JSON pointer mapping → Monaco Editor line highlight).
- PR 2: Fix Handle Misalignment (ResizeObserver integration for dynamic node dimension updates).

Milestone 2 (Weeks 3-4): UI/UX Overhaul for Navigation

- PR 3: Edge-based Navigation Controls (Custom `NavigableEdge` with `fitView` panning buttons).
- PR 4: Distinct Visualization for Multi-Type Schemas (Array type detection + CSS gradient styling).

Milestone 3 (Weeks 5-6): Rendering Algorithms

- PR 5: Topologically Sorted Rendering of `$defs` (Kahn's Algorithm with cycle fallback).
- PR 6: Edge Collision Resolution (Orthogonal constraint routing using `SmoothStepEdge` and node bounding boxes).

Milestone 4 (Weeks 7-8): Utility & Polish (Final Evaluation Phase)

- PR 7: File Upload Support (HTML5 `FileReader` integration).
- PR 8: Downloadable Visualization Output (html-to-image integration for `.react-flow__viewport`).

Final Delivery: Comprehensive documentation of all new UI features and an updated `README.md`.

Availability & Commitments

Expected Time Commitment: 90 Hours (Small Project) My Planned Commitment: 15-20 hours per week (totaling ~120-160 hours across the core 8-week coding period) to ensure all features are polished and extensively tested.

- Timezone: IST (UTC +5:30)
- Daily Schedule: I am most active between 5:00 PM – 2:00 AM IST.
- Mentor Overlap: I will actively align my schedule to ensure at least 2-3 hours of direct overlap with my mentors' working hours for quick unblocking, code reviews, and pair programming if necessary.
- Other Commitments: I have no conflicting internships, full-time jobs, or major vacations planned during the GSoC coding period (May- August). I will be entirely focused on JSON Schema Studio.

Appreciation & Final Note

I want to express my sincere appreciation to @AgniveshChaubey, ioflux-org and the json-schema-org community. The codebase is incredibly well-structured, relying heavily on modern standards like `@hyperjump/json-schema and ReactFlow`, which made diving into the architecture a genuinely enjoyable experience.

Before writing this proposal, I didn't just read the documentation, I spent weeks digging into the internal AST parsing, tracing how ProcessAST builds nodes, and pushing actual bug fixes and enhancements to main.

I did this because I care deeply about developer experience tools. Developers spend hours debugging nested, 1000-line JSON schemas, and JSON Schema Studio completely eliminates that friction.

My goal for GSoC 2026 isn't just to complete an internship; it is to implement these 8 features to make the tool production-ready for massive, enterprise-level schemas. I intend to remain an active maintainer long after the summer ends, helping review PRs, triaging issues, and supporting the next wave of open-source contributors.

Thank you for your time and consideration!